

Kit IA

Configurer un projet pour travailler avec Claude Code —
et savoir ce que la configuration ne peut pas faire.

Ce kit ne parle pas de *prompts*. Il parle de **fichiers** — ceux qu'on pose une fois dans un dépôt et qui changent le comportement de l'agent pour tous ceux qui y travaillent ensuite, y compris vous dans six mois.

Les exemples viennent tous du projet SupportDesk. Ils sont réels : vous pouvez les ouvrir.

1 Le problème qu'on essaie de résoudre

Un agent qui découvre votre dépôt ne sait pas qu'un client ne doit pas voir une note interne. Il ne sait pas que vous êtes en Spring Boot 4.1 et que le starter s'appelle `spring-boot-starter-webmvc` et non `-web`. Il ne sait pas que `ddl-auto` doit rester à `validate` parce que c'est Flyway qui fait le schéma.

Il va donc produire du code **plausible**. C'est le mot important : pas faux au premier regard, pas signalé par le compilateur, et faux quand même.

LE PRINCIPE

Configurer un agent, ce n'est pas lui apprendre à programmer. C'est lui apprendre *votre* projet — ses versions, ses règles, ses interdits — pour qu'il arrête d'inventer ce qu'il ne sait pas.

2 Les cinq leviers, et leur force réelle

FICHIER	QUAND IL AGIT	FORCE
<code>CLAUDE.md</code>	à chaque tour de conversation	une consigne — l'agent peut dériver sur une longue session
<code>.claude/rules/</code>	sur les fichiers désignés	une consigne, mais qui arrive au bon moment
<code>.claude/skills/</code>	quand elle est invoquée	une méthode — sans effet si personne ne la charge
<code>.claude/agents/</code>	quand on lance le subagent	un regard neuf, sans le contexte qui a produit le code
hooks	automatiquement	une barrière — s'exécute que l'agent le veuille ou non

À NE JAMAIS CONFONDRE

Les quatre premiers sont des *consignes*. Le cinquième est une *barrière*. Si quelque chose doit être garanti — un test qui passe, un secret qui ne part pas — ce ne peut pas être une consigne. Réservez les hooks à ça, et n'en mettez pas dix.

3 Écrire un CLAUDE.md qui sert

C'est le fichier le plus lu et le plus mal écrit. Trois défauts reviennent toujours : il raconte l'architecture (l'agent la lit dans le code), il répète des évidences (« écris du code propre »), et il fait douze pages (personne ne suit douze pages, l'agent non plus).

Ce qui doit y figurer

- **Les versions exactes.** C'est ce qui empêche l'invention d'API.
- **Les interdits, formulés comme des interdits.** « *N'expose jamais une entité JPA dans un contrôleur.* »
- **La conduite à tenir dans le doute.** C'est la règle la plus utile de toutes.

Deux règles du `CLAUDE.md` de SupportDesk, à recopier telles quelles dans vos projets :

Si tu n'es pas sûr d'une propriété de configuration ou d'un nom d'artefact, dis-le au lieu de l'inventer.

Ne dis jamais « c'est fait ». Dis « la vérification passe » et montre la commande exécutée avec sa sortie.

POURQUOI LA SECONDE CHANGE TOUT

« C'est fait » est invérifiable. Une commande et sa sortie sont vérifiables par n'importe qui, y compris six mois plus tard. Cette règle ne rend pas l'agent plus compétent — elle rend son travail *auditable*, ce qui vaut mieux.

Le test du fichier

Relisez votre `CLAUDE.md` et supprimez toute ligne dont la violation ne se verrait pas dans une revue de code. Ce qui reste est utile.

4 Les règles conditionnelles

Tout ne mérite pas d'être chargé en permanence. Une règle qui ne concerne que les fichiers de sécurité n'a rien à faire dans le contexte quand on modifie une feuille de style.

`.claude/rules/securite.md`, sur SupportDesk, ne se charge que sur les fichiers sensibles. Extrait :

1. Passe en mode plan. Aucune modification directe dans ces fichiers sans plan approuvé.
2. Toute méthode qui renvoie une donnée appartenant à quelqu'un doit vérifier le propriétaire, explicitement, dans le code.
3. La référence client vient du jeton, jamais de la requête.
4. Aucun secret en dur.
5. CORS : origines listées explicitement. Jamais * avec credentials.

La règle 1 est la plus sous-estimée : elle impose **un plan avant toute modification** sur ces fichiers. Vous relisez une intention en dix lignes plutôt qu'un diff de deux cents.

5 Les compétences (skills)

Une skill est une **méthode**, écrite une fois. Elle transforme une demande floue en procédure. Comparez :

SANS SKILL

« Est-ce que c'est sécurisé ? »

une réponse bien écrite, plausible, non reproductible

AVEC SKILL

« Charge revue-owasp et audite ce fichier. »

cinq questions dans l'ordre, avec des numéros de ligne

Une bonne skill tient sur une page et se termine par un **format de sortie imposé**. C'est ce dernier point qui fait la différence : sans lui, l'agent rédige.

Termine par un verdict : BLOQUANT, À CORRIGER, ou RAS.
Si tu écris RAS, dis explicitement ce que tu as vérifié pour arriver à cette conclusion – un RAS sans justification ne vaut rien.

6 Le relecteur qui n'a pas écrit le code

Un agent qui relit sa propre production a déjà décidé qu'elle était bonne. Il n'a aucune raison de changer d'avis, et il n'en changera pas.

Un **subagent** démarre dans un contexte vide. Il n'a pas vu la conversation, pas vu les hésitations, pas vu le compromis accepté vingt minutes plus tôt. Il voit le code.

```
---
name: code-reviewer
description: Relit un diff à froid, sans le contexte de la session
            qui l'a produit. À invoquer avant toute fusion.
---

Tu relis un diff que tu n'as pas écrit. Ne cherche pas à comprendre
l'intention. Pars du code tel qu'il est.
```

L'ORDRE QUI MARCHE

L'agent principal écrit. Le subagent relit. Vous arbitrez. Trois rôles, et le troisième ne se délègue pas.

7 Les hooks — la seule vraie barrière

Un hook est une commande shell exécutée automatiquement autour d'un outil. Il ne demande pas l'avis de l'agent.

Les usages qui le justifient :

- lancer le formateur après chaque écriture de fichier ;
- refuser une modification qui introduit un motif interdit ;
- relancer les tests d'un module quand il est touché.

VÉRIFIEZ LA SYNTAXE, NE LA RECOPIEZ PAS DE MÉMOIRE

La configuration des hooks a évolué d'une version à l'autre de Claude Code. Lancez `/hooks` dans Claude Code pour voir ce que *votre* version attend, puis

DÉCLENCHÉZ-LE POUR DE VRAI

en modifiant un fichier. Un hook qu'on n'a jamais vu s'exécuter n'existe pas.

8 Le mode plan

Le levier le moins configuré et le plus rentable. Sur un sujet sensible, vous ne voulez pas relire un diff : vous voulez relire une **intention**.

Un plan de dix lignes se conteste en une minute. Un diff de trois cents lignes se survole, et on approuve.

Sur SupportDesk, la règle est écrite noir sur blanc : toute fonctionnalité passe d'abord par un fichier `prompts/NNN-nom.md`, et le plan doit contenir une section « **impact sécurité** » nommant le fichier et la méthode touchés. Obliger à nommer la méthode, c'est obliger à l'avoir cherchée.

9 Ce qui ne marche pas

L'ERREUR	POURQUOI
Un <code>CLAUDE.md</code> de douze pages	Il dilue. Les règles importantes se noient dans les évidences.
Y décrire l'architecture	L'agent lit le code mieux que votre paragraphe, et le paragraphe devient faux avant le code.
« Sois rigoureux », « fais attention »	Invérifiable, donc sans effet. Une règle utile se constate.
Un hook pour chaque envie	Ils s'exécutent tous, à chaque fois. La session devient lente et vous les désactivez tous.
Croire un audit sans numéros de ligne	C'est une rédaction. Vérifiez deux citations au hasard, toujours.
Demander à l'agent s'il a bien fait	Il répondra oui. Demandez la commande et sa sortie.

10 Ce qu'un agent ne trouvera jamais seul

IL TROUVE	IL NE TROUVE PAS
✓ une API qui n'existe pas dans votre version	✗ que ce ticket appartient à quelqu'un d'autre
✓ un secret en dur	✗ qu'un client ne doit pas voir une note interne
✓ un CORS en <code>*</code>	✗ qu'un agent peut, et qu'un client non
✓ un test sans assertion	✗ vos règles métier

La colonne de droite, c'est exactement le n° 1 du classement OWASP API — l'autorisation au niveau objet. Aucun outil générique ne peut la détecter, parce qu'il faudrait connaître votre métier.

LA FORMULE À RETENIR

Un agent bien configuré ne devient pas un expert en sécurité. Il devient un relecteur qui connaît vos règles et ne se fatigue jamais. C'est déjà beaucoup — à condition de ne pas lui demander l'autre chose.

11 Démarrer sur un projet existant

Trente minutes, dans cet ordre :

1. **Les versions.** Ouvrez `pom.xml` et `package.json`, recopiez les versions exactes dans `CLAUDE.md`. C'est le geste le plus rentable des trente minutes.
2. **Trois interdictions.** Ce qui vous ferait refuser une *pull request*. Formulez-les comme des interdictions, pas comme des vœux.
3. **La règle du doute.** « *Si tu n'es pas sûr, dis-le au lieu d'inventer.* »
4. **La règle de la preuve.** « *Montre la commande et sa sortie.* »
5. **Une règle conditionnelle** sur vos fichiers sensibles.
6. **Un subagent relecteur**, même minimal.
7. **Un seul hook**, celui qui garantit ce qui doit l'être. Testez-le.

Puis laissez-le vivre : chaque fois que l'agent se trompe de la même façon deux fois, la troisième correction va dans `CLAUDE.md`. Un fichier de configuration se construit par sédimentation, pas d'un seul jet.

SupportDesk — formation full-stack Angular / Spring Boot · Jour 4, bonus.

Les exemples cités sont extraits du dépôt du projet : `CLAUDE.md`, `.claude/rules/securite.md`, `.claude/skills/revue-owasp/SKILL.md`, `.claude/agents/code-reviewer.md`.

Vérifiez la syntaxe des hooks avec `/hooks` : elle dépend de votre version de Claude Code.